

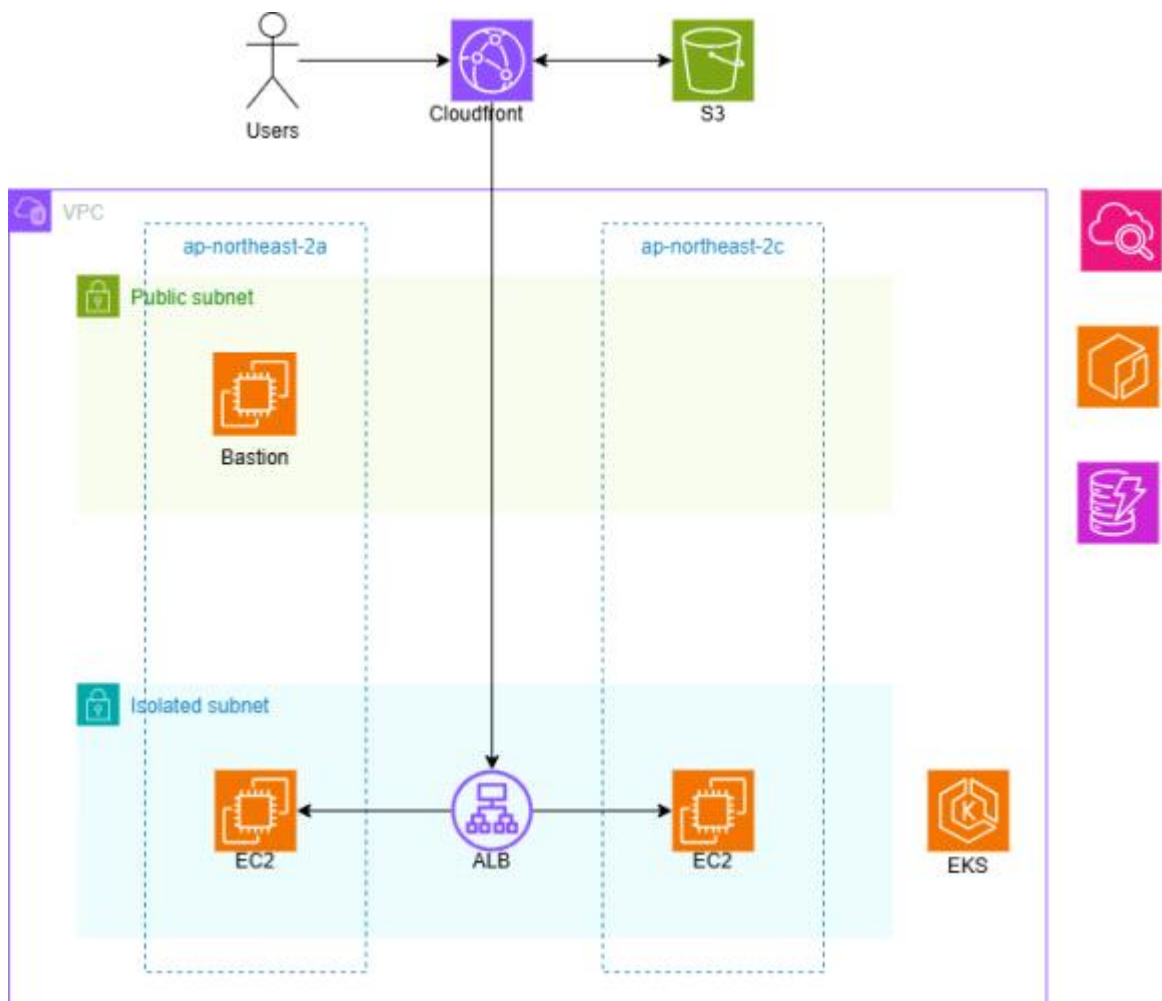
2026년도 전국기능경기대회 과제

직 종 명	클라우드컴퓨팅	과제명	Solution Architecture	과제번호	제1과제
경기시간	4시간	비번호		심사위원 확 인	(인)

1. 요구사항

당신은 WorldPay에서 DevOps 엔지니어로 근무하고 있습니다. WorldPay는 콘서트 예약 서비스를 SaaS 형태로 외부 파트너에게 제공하고자 하며, 예약 요청을 받아 데이터베이스에 저장하는 애플리케이션을 운영하고 있습니다. 해당 서비스를 안정적으로 제공하기 위해 컨테이너 기반 환경과 Kubernetes를 도입하고, 보안성, 고가용성, 운영효율성 등을 고려하여 인프라를 설계하고 구축하세요.

다이어그램



Software Stack

AWS	개발언어/프레임워크
- VPC	- golang/gin
- EC2	- Prometheus
- ELB	- Grafana
- EKS	- Fluent Bit
- ECR	
- Dynamodb	
- CloudWatch	
- IAM	
- KMS	
- CloudFront	
- S3	

2. 선수 유의사항

※ 다음 유의사항을 고려하여 요구사항을 완성하시오.

- 1) 기계 및 공구 등의 사용 시 안전에 유의하시고, 필요 시 안전장비 및 복장 등을 착용하여 사고를 예방하여 주시기 바랍니다.
- 2) 작업 중 화상, 감전, 찰과상 등 안전사고 예방에 유의하시고, 공구나 작업도구 사용 시 안전 보호구 착용 등 안전수칙을 준수하시기 바랍니다.
- 3) 작업 중 공구의 사용에 주의하고, 안전수칙을 준수하여 사고를 예방하여 주시기 바랍니다.
- 4) 경기 시작 전 가벼운 스트레칭 등으로 긴장을 풀어주시고, 작업도구의 사용 시 안전에 주의하십시오.
- 5) 문제에 제시된 괄호박스 < > 는 변수를 뜻하므로 선수가 적절히 변경하여 사용해야 합니다.
- 6) 문제 풀이와 채점의 효율을 위해 Security Group의 80/443 Outbound는 Anyopen하여 사용할 수 있도록 합니다.
- 7) Bastion EC2는 채점 시 사용되기 때문에 종료되거나 연결 문제, 권한 문제 등으로 발생할 수 있는 불이익을 받지 않도록 주의하시기를 바랍니다.
- 8) 모든 리소스는 서울(ap-northeast-2) 리전에 구성합니다.
- 9) 제공자료는 수정 없이 사용합니다. 제공자료를 수정해서 사용하면 불이익을 받을 수 있습니다.
- 10) 문제에서 주어지지 않는 값들은 AWS Well-Architected Framework 6 pillars를 기준으로 적절한 값을 설정해야 합니다.
- 11) 불필요한 리소스를 생성한 경우, 감점의 요인이 될 수 있습니다. (e.g. VPC 추가 생성)
- 12) 모든 리소스의 이름, 태그, 변수와 변수는 대소문자를 구분합니다.

3. VPC

아래의 reference01의 표를 참조하여 vpc를 생성합니다. 각 태그의 뒤에 있는 알파벳은 Availability Zone을 의미합니다.

4. Bastion Server

채점을 위해 AWS EC2를 사용하여 Bastion Server를 생성합니다. 채점 중 ip가 갑작스럽게 변경되는 상황이 없게끔 재시작시에도 ip는 변경되지 않게 구성하여야 합니다. worldpay-public-subnet-a subnet에서 실행되어야 합니다. 채점 시 Administrator에 준하는 IAM 권한을 부여하여 권한 문제가 발생하지 않도록 합니다. 또한 Bastion EC2는 Password 인증을 통해서만 접근이 가능하도록 구성해야 합니다. SSH를 접근에 문제가 생길 경우 불이익을 받을 수 있으니 주의합니다.

- Instance Name Tag : worldpay-bastion
- Machine Image : Amazon Linux 2023
- Instance Type : t3.small
- Login Password : worldpay2026!
- Required Package : awscli, curl, jq, eksctl, kubectl

5. Application

Book 애플리케이션 총 1개의 바이너리가 지급됩니다. 해당 애플리케이션은 AWS_REGION, TABLE_NAME이 필요합니다. (애플리케이션의 API는 Reference02 참조) 애플리케이션 실행 시 바인딩 되는 포트는 8080입니다. 애플리케이션은 /health 경로에서 애플리케이션 상태를 확인합니다.

6. Database

Book 애플리케이션을 안정적으로 저장하기 위해 DynamoDB를 구성합니다. 트래픽 변동에 유연하게 대응할 수 있도록 사용한 만큼만 비용이 청구되도록 구성합니다. 데이터 보호를 위해 고객 관리형 암호화 키를 생성하여 테이블에 적용하고, 장애 발생 시 복구가 가능하도록 설정합니다. 운영 중 액세스 패턴을 분석하기 위해 Contributor Insights를 활성화하고, 실수로 인한 테이블 삭제를 방지하기 위해 삭제 보호를 활성화합니다.

- Table Name: Concerts
- Partition key : booking_id (S)
- Encryption: alias/worldpay-db-key

7. S3 hosting

제공된 정적 파일을 S3를 활용하여 호스팅합니다. 버킷에는 정적 파일(index.html, main.jpeg) 만 존재해야 합니다. 버킷은 외부에서 직접 접근할 수 없도록 퍼블릭 액세스를

모두 차단하고, CloudFront를 통해서만 객체에 접근할 수 있도록 구성합니다. 데이터 보호를 위해 고객 관리형 암호화 키를 생성하여 버킷의 기본 암호화에 적용합니다.

- Bucket Name : worldpay-bucket-<계정 아이디>
- Encryption: alias/worldpay-s3-key

8. Container Registry

제공된 Book 애플리케이션 바이너리를 컨테이너 이미지로 빌드하여 ECR에 업로드합니다. 이미지는 불필요한 패키지를 포함하지 않도록 경량화하여 5MB이하로 구성하고, v1.0.0태그를 지정하여 업로드합니다. 이미지 업로드 시 자동으로 취약점을 스캔하도록 구성합니다.

- ECR repository Name : worldpay-book

9. Container Orchestration

Book 애플리케이션의 안정적인 운영을 위해 AWS EKS 기반의 컨테이너 환경을 구성합니다. 보안 요구사항에 따라 EKS Cluster의 Kubernetes API Server는 Public Endpoint를 비활성화하고 Private Endpoint만 활성화하여 Full Private Cluster로 운영합니다.

- EKS Cluster Name : worldpay-cluster
- EKS Cluster Version : 1.35
- Namespace: worldpay

Managed Nodegroup

모든 워크로드를 Managed Nodegroup을 통해 Isolated Subnet에서 실행되며, 서로 다른 가용 영역에 최소 2대 이상의 Node를 배치하여 고가용성을 확보해야 합니다.

- NodeGroup Name : worldpay-nodegroup
- Node Instance Name Tag : worldpay-node
- Node Instance Type : t3.large

Application Deployments

Book 애플리케이션은 worldpay Namespace에 Deployment로 배포합니다. DynamoDB 접근 권한은 EKS Pod Identity를 활용하여 ServiceAccount에 IAM Role을 연결하는 방식으로 부여합니다.

- Book Deployment Name: book-deploy
- ServiceAccount Name: book-sa

10. Observability

EKS 클러스터에서 운영되는 Book 애플리케이션의 메트릭과 로그를 효과적으로 모니터링하기 위해 Prometheus, Grafana, Fluentbit를 구성합니다.

FluentBit

Book 애플리케이션 Pod에서 발생하는 로그를 Fluentbit를 통해 수집하여 CloudWatch Logs로 전송합니다. Fluentbit는 DaemonSet으로 배포되며, worldpay 네임스페이스의 로그만 수집 대상으로 합니다. 로그 전송 시 Kubernetes metadata를 포함하여야 합니다. (로그 포맷 Reference03 참조)

- Namespace: logging
- DaemonSet Name : worldpay-fluentbit
- Log Group: /worldpay/application
- Log Stream Prefix : book-xxxx
- Log Retention: 7 days

Prometheus

클러스터 및 Pod 메트릭 수집을 위해 Prometheus를 사용합니다. 고가용성을 위해 서로 다른 가용 영역에 독립적인 Prometheus 환경을 구성합니다. 각 Prometheus는 클러스터 전체 메트릭을 동시에 수집해야 하며, 특정 Prometheus 또는 Availability Zone에 장애가 발생하더라도 메트릭 조회는 지속 가능해야 합니다.

- Namespace: monitoring

Grafana

수집된 메트릭과 로그를 시각화하기 위해 Grafana를 구성합니다. Grafana는 외부에서 접근 가능해야 합니다.

- Namespace: monitoring
- username: admin
- password: worldpay2026!

Grafana Datasources

Grafana에는 Prometheus와 CloudWatch Logs 두 개의 데이터소스가 구성되어 있어야 합니다.

- Prometheus Datasource name: worldpay-prometheus (클러스터 및 Pod 메트릭 조회)
- CloudWatch logs Datasource name: worldpay-logs (Book 애플리케이션 로그 조회)

Grafana Dashboard

Grafana에 다음 정보를 확인할 수 있는 대시보드가 구성되어야 합니다. /v1/book API를 호출 시, 대시보드에서 해당 요청에 대한 메트릭 변화와 로그가 1분 이내에 확인되어야 합니다.

- Dashboard Name: worldpay-dashboard
- Pod CPU 사용량
 - Panel Name : POD_CPU
 - Book 애플리케이션 Pod의 CPU 사용량 메트릭 포함
 - Datasource : worldpay-prometheus

- Pod Memory 사용량
 - Panel Name : POD_MEMORY
 - Book 애플리케이션 Pod의 Memory 사용량 메트릭 포함
 - Datasource : worldpay-prometheus
- Application logs
 - Panel Name: APP_LOGS
 - Log Group /worldpay/application의 Book 애플리케이션 로그를 조회
 - Datasource : worldpay-logs
- Network Capacity
 - Panel Name: NETWORK_CAPACITY
 - 각 Availability Zone의 네트워크 할당 가능 상태 모니터링
 - Datasource : worldpay-prometheus

11. Load Balancer

EKS 클러스터에서 운영되는 애플리케이션에 대한 트래픽을 라우팅하기 위해 ALB를 구성합니다.

Book Application ALB

Book 애플리케이션은 Internal ALB를 통해 접근됩니다. ALB는 Isolated Subnet에 위치하며 외부에서 직접 접근할 수 없습니다. 사용자는 CloudFront를 통해서만 Book 애플리케이션에 접근할 수 있습니다.

- Load Balancer Name : book-alb
- Load Balancer Scheme : internal
- Load Balancer Type : Application Load Balancer
- Load Balancer Listen : HTTP 80

Grafana ALB

Grafana는 ALB 통해 외부에서 접근 가능해야 합니다.

- Load Balancer Name : grafana-alb
- Load Balancer Scheme : internet-facing
- Load Balancer Listen : HTTP 80

12. Cloudfront

정적 웹페이지와 Book 애플리케이션 API를 단일 Cloudfront Distribution을 통해 서비스합니다. S3 Origin은 OAC를 사용하여 CloudFront에서만 접근 가능하도록 구성합니다. Book 애플리케이션으로의 API 요청은 캐싱 없이 Internal ALB로 전달되어야 하며, API 요청 시 Origin까지 정상적으로 전달되어야 합니다.

- Distribution name : worldpay-cdn

- OAC Name: worldpay-s3-oac
- Default Root Object : index.html

• Reference01

- Worldpay VPC -

Name	CIDR
worldpay-vpc	10.0.0.0/16

- Worldpay Subnets -

Name	CIDR	Description
worldpay-public-subnet-a	10.0.0.0/24	Internet access
worldpay-public-subnet-c	10.0.1.0/24	Internet access
worldpay-isolated-subnet-a	10.0.2.0/24	No internet access
worldpay-isolated-subnet-c	10.0.3.0/24	No internet access

Reference02

- Application -

API	Request	Response	Description
POST /v1/book	application/json - client_id: Client ID(String) - username: 구매자 이름(String) - email: 구매자 이메일(String) - concert_name: 콘서트(String) Sample: {"client_id": "C001", "username": "Alice", "email": "kim@example.com", "concert_name": "Seoul2025"}	Code 200 - booking_id: 유니크 예약 ID (String) Sample: {"booking_id": "C2011YY"}	HTTP 요청의 Body에 있는 필드 값과 created_at 필드가 DynamoDB 테이블에 저장됩니다.

Reference03

```
{
  "log": "2025-06-14T07:21:30Z INFO POST /v1/book 200",
  "kubernetes": {
    "pod_name": "book-deploy-678956446d-vwtx5",
    "namespace_name": "worldpay",
    "labels": {
      "app": "book",
      "pod-template-hash": "678956446d"
    },
    "host": "ip-10-0-2-160.ap-northeast-2.compute.internal",
    "pod_ip": "10.0.2.186"
  }
}
```